Codeer Agent Skill 客戶安裝指南

這份指南協助你完成 Codeer CLI 與 Codeer Agent Skill 安裝。

重點不是避開 Web UI，而是讓 Codex / Claude Code 能參與完整的 Codeer Agent 生命週期：從資料整理、Agent 規劃、Behavior Cases、Production History 分析，到下一輪迭代。

本指南的本機專案與 Skill 安裝命令需要 Codeer CLI 0.1.17 或更新版本， 以及相配的 codeer-agent skill。CLI 與 Skill 分開安裝及更新。

先分清楚：CLI 和 Skill 各自負責什麼？

Codeer CLI

Codeer CLI 是安裝在本機的 codeer 指令工具。

它負責：

- 登入 Codeer
- 連接 Codeer API
- 執行 codeer agent、codeer eval、codeer history、codeer kb 等指令
- 驗證 API key、workspace、agent 權限是否正常
- 使用 codeer skill 安裝、列出或移除 Codeer Skills（CLI 0.1.17 起）

Codeer Agent Skill

Codeer Agent Skill 是安裝到 Codex / Claude Code 的工作說明。

它負責讓 assistant 知道：

- Codeer Agent 生命週期怎麼走
- 如何從 KB / 檔案討論並建立 Agent v0
- 如何產生 Behavior Cases
- 如何分析 Production History / Negative Feedback
- 什麼時候要先讓你看會改哪些內容，再由你確認是否套用或發布

Codeer Agent Skill 可以發揮什麼威力

1. 從 KB / 檔案開始規劃並創建 Agent v0

把文件、FAQ、SOP、產品資料或資料夾交給 Codex / Claude Code，先討論 Agent 範圍、知識邊界與初版設計。

2. 產生 Behavior Cases

先選擇或設計 Tester、確認它能取得必要證據，再建立 Case、Rubric 與配對。 Tester 就是既有 Evaluator / Judge，讓品質討論有具體依據。

3. 根據失敗問題作出調整

失敗不一定代表 prompt 錯；也可能是 KB、Agent、retrieval、Standard / Rubric 需要修正。

4. 分析 Production History / Negative Feedback

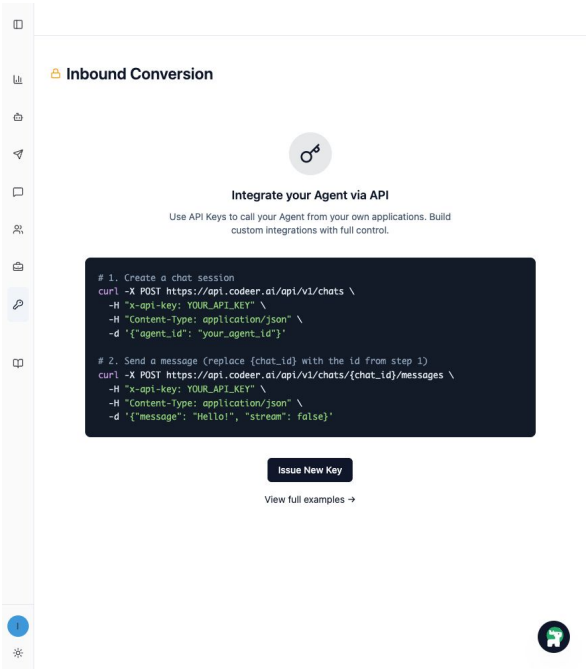
從真實對話和負評中找出模式、缺口與高風險場景，讓下一輪改善有證據。

5. 協助你開始下一輪的迭代

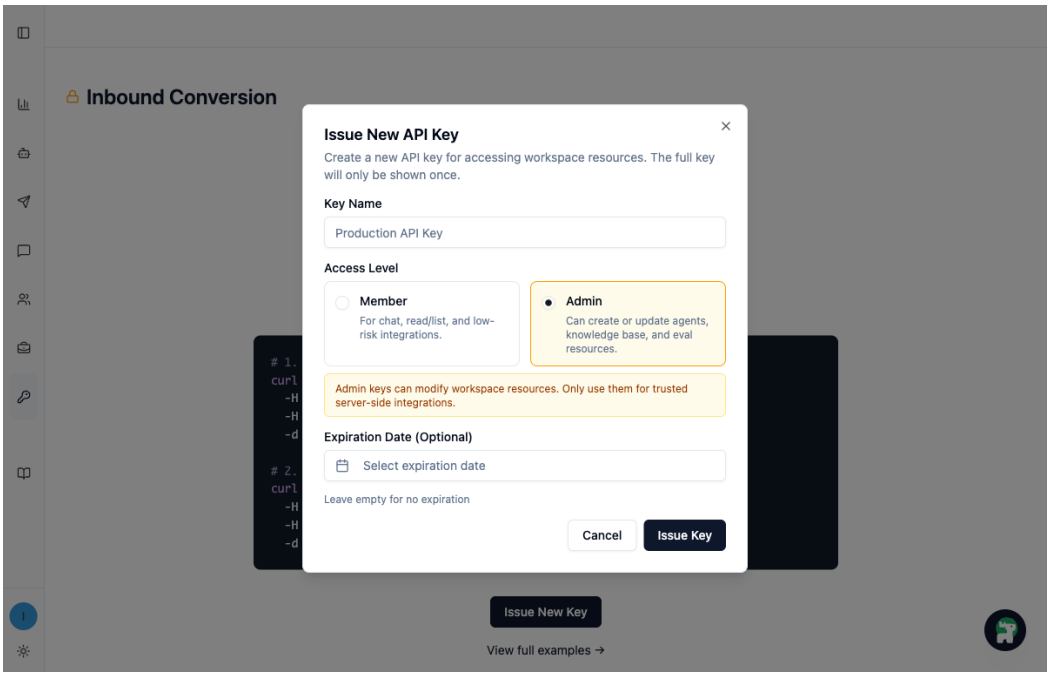
先整理問題、提出修正、預覽會改哪些內容，再由你確認是否套用或發布。

Step 1：在 Codeer 建立 API Key

請使用目標 workspace 的 Admin API key。Skill 會透過 CLI 做 Agent、 KB、Behavior Cases 和 History 相關工作，因此初次設定建議使用 Admin 權限。



在 workspace 頁面左側打開 API Keys，點選 Issue New Key。



輸入容易辨識的名稱，例如 Codex 或 Claude Code，Access level 選 Admin。

完整 API key 只會顯示一次。請立即存到安全位置，不要貼進聊天、prompt、 Git repo、專案 .env 或 .claude/settings.json。

Step 2：安裝 Codeer CLI

Codeer Agent Skill 會透過本機 codeer CLI 操作 Codeer API。建議使用 pipx 安裝，讓 CLI 跟專案 Python 環境分開，比較容易維護。

安裝 CLI：

```
pipx install 'codeer-cli>=0.1.17'
```

確認指令可用：

```
codeer --help
codeer skill --help
codeer init --help
```

如果 PyPI 尚未提供符合版本的套件，可從包含這些命令的版本採用 原始碼安裝。已安裝舊版時， 先執行 pipx upgrade codeer-cli 再核對命令；pipx list 可查看套件版本。 只更新 Skill 不會讓舊 CLI 增加新命令。

如果沒有 pipx：

```
python -m pip install --user pipx
python -m pipx ensurepath
```

重開 terminal 後，再執行：

```
pipx install 'codeer-cli>=0.1.17'
```

維護更新：

```
pipx upgrade codeer-cli
codeer check
```

不建議把 python -m pip install codeer-cli 當作主要安裝方式。它可以作為 fallback，但 pipx 較不容易受到專案套件版本影響。

Step 3：設定本機 CLI Profile

Profile 會把 API key 存在使用者層級的 Codeer CLI config，不需要放進 Codex 或 Claude Code 的設定檔。

下面的 work 只是範例 profile 名稱。你可以換成自己容易辨識的名字，例如 codeer、prod、client-a 或 support-agent。如果你改了名稱，後面的 codeer profile use ... 也要使用同一個名字。

新增 profile：

```
codeer profile add work
```

CLI 會請你貼上 API key，輸入時不會顯示在畫面上。

選用 profile：

```
codeer profile use work
```

檢查 CLI 與 API 是否可用：

```
codeer check
codeer agent list
```

這只能確認 CLI 和 API credential 正常，還不是 Skill 驗證。

如果專案只有一個預設 Agent，可以選填：

```
export CODEER_AGENT_ID=<agent-id>
```

不要把 CODEER_API_KEY 放進 .claude/settings.json、專案 .env、聊天訊息 或 Git commit。

Step 4：安裝 Codeer Agent Skill

使用 Codeer CLI 0.1.17 或更新版本安裝。安裝 Skill 不需要 Codeer API key； 操作 Codeer 伺服器時才需要前面設定的 profile。

Claude Code

```
codeer skill install codeer-agent --target claude
```

Codex

```
codeer skill install codeer-agent --target codex
```

預設安裝到使用者層級：Codex 使用 ~/.agents/skills/codeer-agent， Claude Code 使用 ~/.claude/skills/codeer-agent。加上 --scope project 會改成目前目錄下的 .agents/skills/ 或 .claude/skills/。

使用 codeer skill list 查看安裝狀態。--dry-run 會下載並驗證來源、顯示 安裝目的地，但不寫入安裝檔案； --ref <tag-or-commit> 可固定 Skill 來源版本， 也接受分支名稱，省略時使用 main。來源版本必須包含 skills/codeer-agent 資料夾，並支援已安裝的 CLI。--ref 只選擇 Skill 檔案，不會安裝或升級 CLI； CLI 另外透過 pipx upgrade codeer-cli 更新。

如果安裝後沒有看到 Skill，請重新啟動 Codex；Claude Code 可執行 /reload-skills。

也可以使用 Codex 內建的 installer：

```
$skill-installer install https://github.com/codeer-ai/codeer-skills/tree/main/skills/codeer-agent
```

這種安裝由原本的 installer 管理，codeer skill 不會接管或移除。

更新與移除 Skill

再次執行 install 就會更新未修改的 CLI 管理套件，不必先移除。 先用 --dry-run 預覽，再安裝 main 的版本；也可用 --ref 選擇指定版本。 Claude Code 請將 --target codex 換成 --target claude；若原本使用 --scope project，以下指令也要加上同一選項。

```
codeer skill install codeer-agent --target codex --dry-run
codeer skill install codeer-agent --target codex
codeer skill install codeer-agent --target codex --ref <tag-or-commit>
```

相同內容會回傳 already_installed，不改動檔案；更新成功會回傳 updated。 預覽會回傳 would_install、would_update 或 already_installed。 更新會先下載並驗證完整新版，再替換舊目錄並清除過期檔案；替換失敗時還原 舊版。若還原也失敗，錯誤訊息會提供保留備份的位置，供手動復原。

內容已修改、手動安裝或透過符號連結管理的目錄會被保留；請先保存自訂內容， 再手動處理。若要移除未修改的 CLI 管理套件：

```
codeer skill uninstall codeer-agent --target codex --dry-run
codeer skill uninstall codeer-agent --target codex
```

移除 CLI 不會一併移除 Skill。

Step 5：驗證 Skill 是否被載入

請在 Codex / Claude Code 中問下面這句。這是在確認 assistant 是否真的知道 Codeer Agent 生命週期，而不是只確認 CLI 能不能連線。

```
Please confirm whether the Codeer Agent Skill is loaded. Explain how you can
help me plan an Agent from KB/files, create behavior cases, analyze production
history, and show me proposed changes before applying them.
```

理想回答應該提到：

- Agent lifecycle
- Behavior Cases
- Production History
- 顯示具體修改預覽並核對授權範圍；已確認的同一批內容可沿用授權，發布另行確認

本機專案工作流程

直接描述想要的 Agent，並提供檔案或連結。例如：「依這些 SOP 建立售後 Agent， 可以查訂單，但退款由人工決定。」coding agent 會先讀資料、說明理解並提出建議， 只詢問意圖不明、來源衝突或新的重要取捨，不要求你重複確認已清楚說過的需求。 它會邊探索邊保存設計與待決事項，方便換對話後接續。你不必學習內部模式名稱、 填寫 Agent 設計模板，或為了記錄明確指示再核准一次文件。

整體流程是：理解需求 → 設計行為與測試 → 預覽並套用已授權的 Draft 修改 → 讀回與審查 → 測試指定版本 → 視結果診斷修正 → 明確授權後發布。 以下命令可由 coding agent 代為操作；設計決策本身不等於遠端寫入授權。

以下命令使用 Step 2 安裝的 CLI 0.1.17 或更新版本。開始前先檢查 codeer init --help；若找不到命令，先回到 CLI 安裝步驟處理版本。

```
codeer init ./support-agent
cd support-agent
codeer status
codeer validate
```

這三個命令不需要憑證或網路。接手既有 Agent 時，先選擇正確 profile，再執行：

```
codeer agent bind prod-customer-a-main --agent <id>
codeer agent use prod-customer-a-main
codeer agent pull --agent <id>
codeer eval pull
codeer agent diff --local
```

一個本機專案對應一個主要 Agent，不會建立遠端 Project。原始 SOP / PDF 放在 sources/documents/，以 sources/index.md 記錄來源；整理後的 KB 放在 kb/。 coding agent 將已確定的設計決策放在 design/agent-design.md，待確認提案放在 design/agent-design.draft.md。logs/YYYY-MM.md 簡記重要理由、狀態與證據連結；既有 log.md 繼續使用，不自動拆檔。 Prompt 的唯一編輯來源是 agent/instructions.md，其他設定放在 agent/settings.json；穩定 Tester / Case / Rubric 配對放在 evals/design.json。

既有專案沿用設定的路徑與紀錄，更新 CLI 不會搬檔或建立第二份設計。 需要選擇新舊設計路徑時，請見專案存放與既有路徑。

直接用日常語言提出需求，coding agent 會選擇 build、cases、audit、apply、eval、improve、publish、observe 八種 Skill 模式，也可以明確指定模式。例如「只診斷這題轉人工失敗」、「套用先前確認的修改並測試， 不要發布」，或「從上週 AI Draft 回饋提出測試候選」。這些是 Skill 模式， 不是 CLI 子命令。既有 Agent 可以從局部診斷開始；匯入現況不等於接受業務規則或品質。

專案套用先取得 --dry-run 預覽，再傳入 --expect <token>。 token 只辨識內容，不代表授權。本機 / 網頁衝突與結果不明都會保留，需先核對再重試。 套用 Draft 不等於發布。每輪測試在 runs/ 保存確切 result IDs；發布會核對有效 語意 audit、已確認驗收範圍與涵蓋該範圍的通過 run，再於 releases/ 保存發布讀回。 缺少工具證據或只有舊結果不能算成功；Judge 分數本身也無法證明操作真的發生。

格式、舊專案接手方式及 API 限制請見 專案契約。原始 History / AI Draft 匯出留在 sources/history/<collection-id>/ 私有目錄；候選測試保留來源，經審查才套用。

Clone、binding 切換、KB 可攜、操作恢復與交付，請依需求查閱 進階專案操作。

Codeer Agent Skill 會怎麼和你協作？

使用你的工作語言即可；它不必和文件或 Agent 對客語言相同。coding agent 會說明 已知目標、具體資料 / 能力缺口、建議下一步與完成條件。缺退款規則時，仍可設計 商品比較與「不編造退款承諾」測試，但不能宣稱已驗證退款資格判定。既定決策會 沿用，不要求你填內部模板或重複確認。

案例可參考真實問題、客戶政策與公開類似情境，並標明來源；別家政策不直接變成 你的答案。較廣的設計可用互補 worker 與獨立審查，小修改不用委派。

一般測試包含結果與初步診斷：資料本來缺少、整理遺漏、檢索未取得、回答誤用或 評分器誤判，各有不同修正。原因不明會提出下一個最小檢查，不把低分都變成 Instructions 規則。你也可以要求只匯出或只診斷單一問題。

既有專案不用重新初始化或搬檔。codeer status --human 提供離線引導； codeer eval inspect-run --run-id <id> 可讀取保存的測試證據，不修改原紀錄、 不重跑模型，也不等於驗收。主要入口一致，KB / 延伸分析可不同；有實質成果的 批次集中一份主要 review，普通查詢不另寫報告。

診斷或修正會保留原始測試證據，方便之後比較。需要封存格式與完整性細節時， 請見測試證據與受控交付。

Skill 的角色是把 Agent 改善流程拆成可以討論、可以確認的步驟。它不只是 幫你下指令，也會協助判斷下一步該改什麼。

- 先了解你的目標、資料來源和目前 Agent 狀態，再提出適合的工作順序。
- 需要建立或調整 Agent、KB、Behavior Cases、Rubrics 時，會先說明它準備 做什麼。
- 真正套用修改前，會先讓你預覽會改哪些內容，或用修改前後差異的方式讓你 確認。
- 發布新版 Agent 前，會先確認測試結果、失敗分析和修改內容都已經被看過。

CLI 常見問題快速排除

codeer command not found

執行：

```
pipx ensurepath
```

然後重開 terminal 再試一次。

codeer check 顯示 401 / 403

401 / 403 不能單獨判定過期，也不能區分憑證遭拒與權限不足。先核對設定來源、API / workspace 及權限；確認 key 無效後，才明確用 codeer profile add <name> 更新相應 profile，再 check。若 CODEER_API_KEY / CODEER_API_BASE 正在覆蓋 profile，須修正或移除變數，單純切換 profile 無法修復。網路中斷、逾時或 5xx 只代表連線尚未驗證，保留原設定並檢查服務。

```
codeer profile current
codeer check
```

看到錯的 workspace

先核對 API base 與實際 key 的 workspace；修正環境覆蓋或選擇正確 workspace 的 profile，再 check。不自動改綁部署：

```
codeer check
```

找不到 Agent

確認 CODEER_AGENT_ID 是否屬於同一個 workspace；不確定時先 unset 後列出 workspace agents：

```
unset CODEER_AGENT_ID
codeer agent list
```

安裝完成後，下一步做什麼？

如果你還沒有 Agent

可以問：

這些是我的 KB / 文件資料夾，請先跟我討論適合建立什麼 Agent v0、需要哪些資料、有哪些風險邊界。

如果你有 Agent，但還沒有 Behavior Cases

可以問：

請根據這個 Agent 的任務，選擇或設計 Tester、確認必要證據，再提出 Case、Rubric 和配對，先讓我審核再套用。

如果你有 Behavior Cases，但有些沒有通過

可以問：

請分析失敗原因，判斷應該修 KB、Agent instructions、retrieval，還是 Standard / Rubric，並先讓我看會改哪些內容。

如果你的 Agent 已經上線

可以問：

請分析 Production History / Negative Feedback，找出常見失敗模式、缺少的測試覆蓋，以及下一輪應優先處理的問題。

如果你準備開始下一輪迭代

可以問：

請根據目前問題、測試結果與歷史紀錄，提出下一版修改計畫；先讓我看證據與會改哪些內容，再決定是否套用或發布。

進階專案操作

需要 Clone 專案、切換遠端 binding、分離共用資源、恢復操作或分享正式證據時， 再查閱對應章節；coding agent 可以依本次目標協助選擇與操作。

專案存放與既有路徑

Agent 專案放在自己的管理 repo，例如 projects/<stable-name>/；root README 可提供索引，共同 AGENTS 保存維護規則。Repo root 的 ignore 也加 .codeer/，保護繼承的 profile 選取；project init 保留父目錄檔案。

現行 project.toml 只有 schema 2，不保存目標或個人 profile；state / resources 等物件的 schema 1 是各自契約，不是舊專案。符合現行格式即可零變更使用；不支援的專案在認證前拒絕，必要時依一次性整理手冊在副本備份、hash 核對與人工審核後處理。

沒有日常 migrate 命令；歷史證據與內嵌設定保持原 bytes，不升級成新版 pass。

上述為新專案預設。既有專案沿用 [paths].design 或舊 [paths].contract 指定的檔案；未指定時，若只有舊 design/behavior-contract.md 或其草稿，便沿用舊位置。

新舊兩組檔案並存時，以 [paths].design 明確選定來源。待確認提案放在選定檔案旁的<stem>.draft.md，不另建第二份設計。

更新 CLI 不會搬移既有檔案；純名稱調整不會使證據過期，設計內容或其他依賴改變時仍需重新驗證。

Clone 與連線選擇

Clone 後沒有 .codeer/ 是正常狀態。先沿用已明確設定的環境變數、最近父目錄 / 專案 profile 或全域預設，以 profile current、check 核對。

只有缺設定才明確執行 profile add <name>，其他平台加 --api-base；未選取則用 profile use。缺失 / 遭拒的 key 應修正實際設定來源；切換 profile 不會消除環境覆蓋。

互動式 add 才安全保存輸入至受限權限的使用者設定，不自動蒐集 / 保存 key。網路中斷、逾時或 5xx 表示尚無法驗證連線；權限不足與原因未明的 401 / 403 都不等於 key 過期。

Binding 是本專案對一個遠端 Agent 的命名映射，不是環境名稱。agent bind beta-customer-a-main --agent <id> 接手既有 Agent；agent bind beta-customer-a-candidate --new-agent 先建立本地映射，之後 apply 才建立遠端 Agent。

同 workspace 可有 main / candidate，多 workspace 可另取 beta-customer-b-main。名稱限專案內唯一、1–64 個英數 / 底線 / 連字號且以英數起頭。

以 agent use <name> 或全域 --binding <name> 選取；agent bindings 離線顯示 API、workspace / Agent 名稱與 ID、選取及未知操作。實體仍用 .codeer/deployments/<name>/，名稱不是平台身分。

全域 --profile <name> 只影響這次命令，涵蓋 check、bind、預覽及 status refresh，不改預設或保存 key。明確 profile 若遇到任何 CODEER_API_KEY 覆蓋，或不同的非空 CODEER_API_BASE，會在認證前拒絕；尾端斜線視為等價。

未用旗標仍沿用環境 / 目錄 / 全域優先序。Binding 不保存 profile 清單；同 API / workspace 的合法 key 可使用，但換連線需重看預覽。

共同遠端寫入仍需操作者協調，本機鎖不提供遠端交易保證。

共用 Tester 與 KB

共用 Tester / KB 本身是合理選擇。只改 candidate 的 Rubric 不會改共用 Tester；共同更新 Tester 時，預覽會列出可讀到的 Agent / Case 影響，影響改變需重新確認。

若只改 candidate 的 Tester，使用 eval tester-fork --tester <local-id> 預覽 / expect：新建 Tester、只接回該 Agent 已映射的 Cases，讀回後換映射。KB 則用 kb upload --fork 建立新 KB / file IDs；仍需核對 / 重建 FAQ，再預覽並套用 Agent 的新引用，才算完成分離。

保留舊資源；中斷用已知 ID reconcile，未知新增結果不盲目重送。

原件與可攜 FAQ

在 resources.json 宣告必要原件與 KB 來源。Agent / Case 附件使用 attachment:<local-id>，知識工具使用 kb:<local-id>[/path]，環境值 / 憑證使用明確的 env:<VARIABLE>，只在 HTTP 邊界解析，不提交憑證。

以 agent attachment 上傳原件，KB 則依宣告的 local ID / 目錄 / glob 上傳；都先預覽，再用已核對的 --expect。必要 PDF / 附件可受控進 Git / 儲存；新專案不加廣泛 PDF ignore，原始客服對話、一般 run / release 留本機。

可攜匯入透過 resource export API 保存 KB 原件、未改寫文字及完整 FAQ。kb import --local-id <id> --kb-id <id> 先預覽再用 expect，分別保存原件、FAQ 定義與來源 / 完整性證據。

目標 KB 上傳或綁定後，kb faq-sync --remote 可採納平台 FAQ 至本機；--keep-local 保存待套用提案，再另外預覽並確認 --apply 才重建。既有 FAQ 必須明確映射 ID，重建文字範圍前須核對目標文字一致；未知新增身分會阻擋重試。

調查後明確提供 ID，可恢復原紀錄的操作，即使較新的本機提案已移除該 FAQ。恢復不會採納其他平台變動，須重新比較並決定。

部分 KB 匯入會保留可取得檔案並逐項記錄缺口；宣告的 FAQ / manifest 必須齊備，才能通過驗證與交付。網站爬取設定先排除並記錄缺口。

API 或原件缺失仍保留缺口，不用文字或來源 URL 代替已核對原件。詳見 KB / FAQ 流程。

接回平台修改

平台有人修改時先 pull 觀察，再以 agent adopt --resource agent --fields description --remote --dry-run 預覽，重複相同命令加 --expect <token> 才採納。--keep-local 保留本地提案為待套用；Case / Tester / 配對明確指定 ID，不依名稱配對。

kb bind 可核對來源檔與 node ID，並採納文字快照。已對齊、待套用與尚未決定分開保存；採納不寫平台，未決欄位阻擋整份資源更新。

任一側改變即須重做預覽；結果不明先恢復，不推送另一環境。

接手既有附件使用 agent attachment --local-id <id> --remote-id <uuid>，先預覽，再用 expect 核對下載 bytes 與本地原件，不重新上傳。已部署 / 接手的目標若要建立新附件或 KB，須明確加 --new；缺少映射不能當成重建理由。

沒有原專案時，匯入的是現況：可下載的附件會核對 bytes，URL 過期 / 原件不可取仍列缺口；KB export 是 Markdown，不是原始 PDF。設計理由、需求確認與前任本機歷史不能反推。

子 Agent 引用保留供唯讀診斷，阻擋專案部署；不遞迴建立或刪除引用。這類匯入不代表完整備份、接受業務規則或品質通過；既有 KB 內容也沒有原地替換 API。

操作紀錄與清理

imports/ 保存遠端觀察，未變 pull 重用快照並更新觀察時間；receipts/ 保存請求、ID、讀回與恢復資訊，不只是 logs。新版省略可衍生的比較副本；固定證據引用的 receipt 在恢復時另建衍生紀錄，保留原路徑 / SHA-256。

不要直接刪 .codeer/ 或 state 來處理未知結果。

清理先用 codeer --binding <name> clean --dry-run 查看檔案、bytes 與保留理由，再同條件 --expect <token>；可加 --older-than 30d。只清所選 binding 已完成、未被引用的機器歷史，JSON / Markdown / 裸路徑 / hash、所有 binding 的 state 與人工 clean --keep <path> --reason <reason> 都會保護內容。

原件、run / release、固定歷史不列候選。CLI 取本機鎖後重驗；直接寫檔的 skill audit / pin、舊 CLI 或 standalone 工具仍需在清理 / 整理期間停止。

本機鎖不保證遠端交易或其他操作者安全。

測試證據與受控交付

首輪比較會保留診斷當時的證據。現行契約 2 的 passed / failed / stale run 用連結與原檔SHA-256 固定；其他狀態仍可能被匯出更新，會在診斷或修正前保存一次原始run.json bytes 與雜湊，不覆寫封存，也不整包複製目錄。

未變的封存驗證後直接沿用；資料不完整仍是不完整，不因此成為驗收通過。舊契約 run 一律唯讀原資訊 / hash，不做新版判讀、refresh 或發布 clearance。

Release 採唯一平面 JSON 檔，保留 pending / unknown / 讀回狀態，也能讀舊目錄。正式交付使用 eval delivery --run-id <id> --release-id <id> --out-dir <新目錄>，再以 eval delivery --verify <目錄> 讀回檢查。

保存確切 result IDs、目標 / 內容指紋及必要 audit / 結果 / trace / 發布證據；缺檔仍為不完整，摘要不能取代證據。不匯出 .codeer/ 或其他試跑；原始證據需受控分享。

此檢查證明離線完整性與身分核對，不是即時平台驗證。